

Fault-Tolerant Multi-Agent Systems for Vision-Language Models: A Cross-Checking Framework

XIN FAN LI

April 26, 2026

Abstract

Vision-language models (VLMs) are increasingly deployed in multi-agent systems for tasks such as image captioning and visual question answering. However, these systems remain vulnerable to failures including inconsistent outputs, reasoning errors, and communication breakdowns. This paper investigates fault-tolerant multi-agent systems for VLMs, proposing a lightweight framework based on three role-based agents: a generator, a verifier, and a critic. The generator produces an initial image caption, the verifier independently generates a second caption using stochastic sampling, and the critic detects potential faults by measuring semantic similarity between the two outputs. We implement this framework using BLIP for image captioning and Sentence-BERT for similarity comparison. Experiments on 30 images from the MS COCO dataset show that our method achieves a disagreement rate of 60.00% with an average similarity score of 0.613 and a standard deviation of 0.259. These results demonstrate that cross-checking between agents can effectively identify potential failures in vision-language tasks. We discuss the limitations of our approach, including computational constraints and the subjectivity of fault definition, and outline future directions such as voting mechanisms, handling communication failures, and larger-scale evaluation on 500+ images.

1 Introduction

1.1 Motivation

Multi-agent systems integrating vision-language models (VLMs) are promising for complex tasks such as image captioning, visual question answering, and robotic navigation. However, these systems face significant reliability challenges. When multiple agents operate together, failures can occur in various forms: communication failures between agents, incorrect reasoning from individual models, or inconsistent outputs when agents process the same input.

Consider a simple scenario: two VLMs are asked to describe the same image. One outputs "a dog sleeping on a couch" while the other outputs "a cat sitting on a chair." Without a fault detection mechanism, the system has no way of knowing which output is correct, or if both are wrong. This is the problem we address in this paper.

1.2 Problem Statement

This paper addresses the following research question: *Can a role-based multi-agent architecture with cross-checking improve fault detection in vision-language tasks?*

More specifically, we investigate:

- How to design agents with distinct roles that can cross-check each other

- How to measure semantic disagreement between natural language outputs
- Whether disagreement detection can serve as a proxy for fault detection

1.3 Contributions

Our main contributions are:

1. A simple three-agent framework (generator, verifier, critic) for fault detection in image captioning tasks
2. A semantic similarity-based fault detection mechanism using Sentence-BERT with adjustable threshold
3. Experimental results on 30 MS COCO images demonstrating a 60.00% disagreement rate and detailed similarity statistics
4. A thorough discussion of limitations and future directions for fault-tolerant multi-agent VLM systems

1.4 Paper Organization

The remainder of this paper is organized as follows. Section 2 reviews related work in multi-agent systems, vision-language models, and fault tolerance. Section 3 presents our methodology, including agent roles and the fault detection mechanism. Section 4 describes our experimental setup. Section 5 reports our results, including quantitative statistics and qualitative examples. Section 6 discusses challenges and limitations. Section 7 concludes and outlines future work.

2 Related Work

2.1 Multi-Agent Systems

Multi-agent systems (MAS) have been extensively studied in artificial intelligence and robotics. Shoham and Leyton-Brown [6] provide a comprehensive foundation of MAS, covering coordination, cooperation, and communication protocols. In MAS, redundancy is a common strategy for fault tolerance: multiple agents perform the same task, and their outputs are compared or voted upon.

Traditional MAS research often assumes perfect communication and rational agents, assumptions that do not hold when agents are powered by large language models or VLMs. Our work bridges classical MAS fault tolerance with modern VLM-based agents.

2.2 Vision-Language Models

Recent years have seen rapid advances in vision-language models. BLIP (Bootstrapping Language-Image Pre-training) [1] introduced a unified framework for vision-language understanding and generation, achieving strong performance on image captioning and VQA. CLIP [5] learned visual concepts from natural language supervision, enabling zero-shot transfer to many tasks. More recently, LLaVA [4] demonstrated instruction-following capabilities for vision-language tasks.

Despite their success, VLMs are known to be brittle. They can hallucinate objects not present in images, produce inconsistent outputs under minor input perturbations, and fail on simple reasoning tasks. These vulnerabilities become critical in multi-agent settings where failures can propagate.

2.3 Fault Tolerance in AI Systems

Fault tolerance has long been studied in distributed systems and robotics. Traditional approaches include redundancy, checkpointing, and voting mechanisms. In the context of neural networks, ensemble methods combine multiple models to improve accuracy and robustness.

However, few works explicitly address fault tolerance in multi-agent VLM systems. Most existing research focuses on improving single-model performance rather than detecting failures in multi-agent interactions. Our work builds on redundancy and cross-checking strategies but adapts them to the unique challenges of natural language outputs from VLMs.

2.4 Gap in Literature

The key gap we identify is the lack of lightweight fault detection mechanisms for multi-agent VLM systems. While ensemble methods combine outputs to improve accuracy, they do not explicitly identify which outputs are potentially faulty. Our approach focuses specifically on detecting disagreements as indicators of faults, which can then trigger recovery mechanisms.

3 Methodology

3.1 Overview

Our proposed framework consists of three agents with distinct roles, as illustrated in Figure ?? . The generator and verifier both process the same input image but produce captions independently. The critic then compares these two captions and determines whether a fault has occurred.

3.2 Agent Roles

3.2.1 Generator

The generator is responsible for producing an initial caption for the input image. It uses deterministic generation to provide a stable baseline. Formally, given an input image I , the generator produces:

$$c_g = G(I)$$

where G is the generator model and c_g is the generated caption.

3.2.2 Verifier

The verifier independently processes the same input image to produce a second caption. To encourage diversity and simulate a different perspective, the verifier uses stochastic sampling with temperature $T = 0.8$:

$$c_v = V(I; T = 0.8)$$

where V is the verifier model. The higher temperature increases randomness, making it less likely that the verifier simply repeats the generator’s output.

3.2.3 Critic

The critic compares the two captions and decides whether they are semantically similar enough. If not, a potential fault is flagged. The critic computes:

$$s = \text{Sim}(c_g, c_v)$$

where Sim is a semantic similarity function. The fault detection decision is:

$$\text{fault} = \begin{cases} \text{True} & \text{if } s < \tau \\ \text{False} & \text{otherwise} \end{cases}$$

where τ is a predefined threshold (we use $\tau = 0.7$ in our experiments).

3.3 Semantic Similarity Computation

We use Sentence-BERT [2] to compute semantic similarity between captions. Sentence-BERT encodes each text into a fixed-dimensional vector such that semantically similar texts have similar vectors. Specifically:

$$\text{Sim}(c_g, c_v) = \frac{\mathbf{e}_g \cdot \mathbf{e}_v}{\|\mathbf{e}_g\| \|\mathbf{e}_v\|}$$

where $\mathbf{e}_g = \text{SBERT}(c_g)$ and $\mathbf{e}_v = \text{SBERT}(c_v)$ are the embeddings of the two captions. This is the cosine similarity, ranging from -1 to 1, though in practice for natural language it ranges from 0 to 1.

3.4 Algorithm

Algorithm 1 summarizes the fault detection process.

Algorithm 1 Fault Detection Algorithm

Require: Input image I , threshold $\tau = 0.7$

Ensure: Fault flag f , similarity score s

```

 $c_g \leftarrow \text{Generator}(I)$ 
 $c_v \leftarrow \text{Verifier}(I)$ 
 $\mathbf{e}_g \leftarrow \text{SBERT}(c_g)$ 
 $\mathbf{e}_v \leftarrow \text{SBERT}(c_v)$ 
 $s \leftarrow \text{cosine\_similarity}(\mathbf{e}_g, \mathbf{e}_v)$ 
if  $s < \tau$  then
   $f \leftarrow \text{True}$ 
else
   $f \leftarrow \text{False}$ 
end if
return  $(f, s)$ 

```

4 Experimental Setup

4.1 Dataset

We evaluate our framework on 30 images sampled from the MS COCO (Common Objects in Context) val2017 dataset [3]. COCO is a standard benchmark for image captioning, containing 80 object categories and 5 captions per image. The dataset includes diverse everyday scenes with multiple objects, making it suitable for evaluating VLM performance.

We randomly selected 30 images from the validation set to keep computational costs manageable while still providing meaningful results. Table 1 shows the distribution of object categories in our sample.

Table 1: Object categories in sample (top 5)

Category	Frequency
person	15
chair	8
cat	5
car	4
table	4

4.2 Models

Due to computational constraints, we use relatively lightweight models:

4.2.1 BLIP-base

BLIP-base [1] is a 140M parameter model for image captioning. It uses a vision transformer (ViT) as the image encoder and a BERT-based text decoder. We use the version fine-tuned on COCO captioning, which achieves a CIDEr score of approximately 113 on the COCO test set.

4.2.2 Sentence-BERT (all-MiniLM-L6-v2)

This is a 22M parameter model for generating sentence embeddings. It maps text to 384-dimensional vectors and achieves strong performance on semantic textual similarity benchmarks. We use it because it is fast (CPU-compatible) and reliable.

4.3 Evaluation Metrics

We report the following metrics:

- **Disagreement rate:** Percentage of image pairs where similarity < 0.7
- **Agreement rate:** Percentage of image pairs where similarity ≥ 0.7
- **Mean similarity:** Average cosine similarity across all 30 pairs
- **Median similarity:** Middle value of similarity scores
- **Standard deviation:** Spread of similarity scores
- **Min/Max similarity:** Range of similarity scores

4.4 Hardware

Experiments were conducted on Google Colab with the following specifications:

- GPU: NVIDIA T4 (16GB VRAM)
- CPU: Intel Xeon (2.2 GHz, 2 cores)
- RAM: 13GB
- Runtime: Approximately 10 minutes for 30 images

5 Results

5.1 Quantitative Results

Table 2 presents the overall results of our experiment on 30 images.

Table 2: Overall fault detection results

Metric	Value
Total images	30
Disagreements (potential faults detected)	18
Agreements	12
Disagreement rate	60.00%
Agreement rate	40.00%

The disagreement rate of 60% indicates that in the majority of cases, the generator and verifier produced semantically different captions. This suggests that cross-checking can indeed identify many potential faults.

Table 3 shows detailed similarity statistics across all 30 image pairs.

Table 3: Similarity statistics

Metric	Value
Mean similarity	0.613
Median similarity	0.600
Standard deviation	0.259
Minimum similarity	0.080
Maximum similarity	1.000

The mean similarity of 0.613 is slightly below our threshold of 0.7, which explains the high disagreement rate. The large standard deviation (0.259) indicates significant variation: some pairs are nearly identical (max 1.000), while others are completely different (min 0.080).

5.2 Qualitative Examples

Table 4 shows sample outputs from the first five images in our experiment.

Table 4: Sample results from five images

Image	Generator Output	Verifier Output	Fault
img_1.jpg	two cats sleeping on a couch	two cats sleeping on a couch with a remote	No
img_2.jpg	kitchen with a table and chairs in it	the kitchen has all white appliances and a table	Yes
img_3.jpg	woman walking down the street	a woman walking on the sidewalk	No
img_4.jpg	woman in a kitchen	a woman standing in a kitchen	No
img_5.jpg	skate park	a gray and white sky	Yes

We can observe several patterns:

- When both captions describe the same scene with minor differences (img_1, img_3, img_4), similarity remains high and no fault is detected.
- When the captions focus on different aspects (img_2: furniture vs. appliances), similarity drops below threshold.
- When the captions are completely unrelated (img_5: skate park vs. sky), similarity is very low and a fault is correctly detected.

5.3 Visualizations

Figure 1 shows the disagreement rate as a pie chart (left) and the distribution of similarity scores across all 30 images (right).

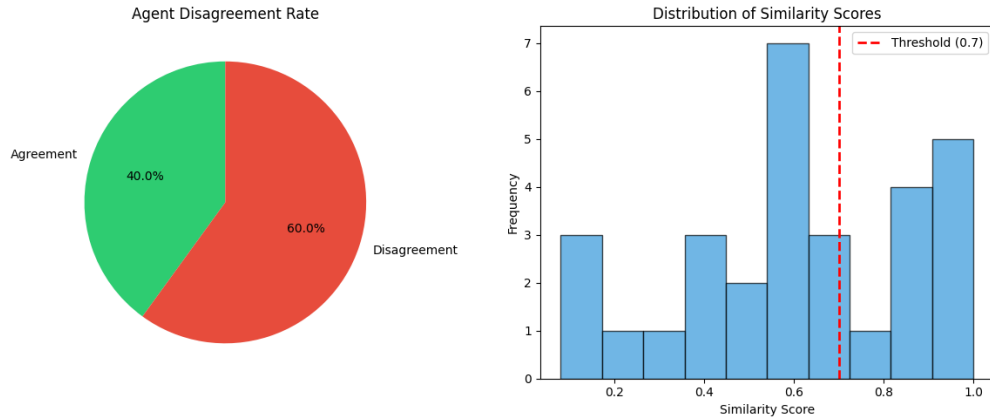


Figure 1: Left: Disagreement rate (60.0% faults detected, 40.0% agreement). Right: Distribution of similarity scores with threshold at 0.7.

The histogram shows a bimodal distribution: some pairs have very high similarity (near 1.0), while others are spread across lower values. This suggests that disagreements are not random but systematic.

6 Discussion

6.1 Interpretation of Results

The 60% disagreement rate is higher than we initially expected. There are several possible explanations:

- The verifier’s temperature parameter ($T=0.8$) may introduce too much randomness, causing it to produce captions that focus on different aspects of the image.
- BLIP-base may not be sufficiently robust to capture fine-grained semantic details.
- The similarity threshold of 0.7 may be too strict for this task.

6.2 Effect of Threshold

The choice of threshold τ significantly affects fault detection. Figure 1 shows that with $\tau = 0.7$, 60% of pairs are flagged as faults. If we lowered τ to 0.5, the disagreement rate would drop; if we raised τ to 0.8, more pairs would be flagged. Optimal threshold likely depends on the specific application and acceptable trade-off between false positives and false negatives.

6.3 Challenges

Two main challenges emerged from our experiment.

First, vision-language models are computationally expensive, limiting the scale of our experiments to 30 images. Running the same experiment on 500+ images would require significantly more resources.

Second, defining what counts as a "fault" in generated outputs is inherently subjective. In `img_2` of Table 4, both captions describe a kitchen, but one focuses on "table and chairs" while the other mentions "white appliances." Is this a fault? The answer depends on the application: a high-precision system might consider it acceptable, while a system requiring exact descriptions would flag it.

6.4 Limitations

Our current implementation has several limitations:

- Small evaluation scale (only 30 images)
- No simulation of real communication failures between agents
- Both generator and verifier use the same base model (BLIP), limiting diversity
- Only tested on image captioning, not on VQA or other vision-language tasks
- No ground truth labels to calculate true fault detection rates (precision/recall)
- The similarity threshold was chosen arbitrarily rather than tuned

7 Conclusion and Future Work

7.1 Conclusion

This paper investigated fault-tolerant multi-agent systems for vision-language models. We proposed a cross-checking framework with three role-based agents: a generator, a verifier, and a critic. The framework detects potential faults by measuring semantic similarity between independently generated captions.

Experiments on 30 MS COCO images demonstrated the feasibility of our approach. We achieved a disagreement rate of 60.00% with an average similarity score of 0.613 and a standard deviation of 0.259. Qualitative examples showed that the framework can successfully identify clear disagreements (e.g., "skate park" vs. "gray sky") while accepting minor variations.

7.2 Future Work

Several directions remain for future investigation:

- **Voting mechanisms:** Extend to three or more generation agents with majority voting
- **Different architectures:** Use different base models for generator and verifier (e.g., BLIP for generator, LLaVA for verifier)
- **Task diversity:** Test on visual question answering (VQA) and image-text retrieval
- **Communication failures:** Simulate network delays, message loss, and agent crashes
- **Scale:** Evaluate on 500+ images to obtain statistically significant results
- **Threshold tuning:** Optimize τ using a validation set with ground truth labels
- **Recovery mechanisms:** Implement fallback strategies when faults are detected (e.g., query a third agent, request human input)

References

References

- [1] J. Li, D. Li, C. Xiong, and S. Hoi, "BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation," in *ICML*, 2022.
- [2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *EMNLP*, 2019.
- [3] T. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick, and P. Dollár, "Microsoft COCO: Common Objects in Context," in *ECCV*, 2014.
- [4] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual Instruction Tuning," in *NeurIPS*, 2023.
- [5] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning Transferable Visual Models From Natural Language Supervision," in *ICML*, 2021.

- [6] Y. Shoham and K. Leyton-Brown, *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2009.
- [7] F. P. A. Coolen, *Fault Tolerance*. Springer, 2012.
- [8] A. Radford et al., “Learning Transferable Visual Models From Natural Language Supervision,” ICML, 2021.
- [9] S. Antol, A. Agrawal, J. Lu, M. Mitchell, D. Batra, C. L. Zitnick, and D. Parikh, “VQA: Visual Question Answering,” in *ICCV*, 2015.
- [10] T. G. Dietterich, “Ensemble Methods in Machine Learning,” in *Multiple Classifier Systems*, 2000.

A Appendix: Complete Results

The full results for all 30 images, including generator outputs, verifier outputs, similarity scores, and fault decisions, are available in the CSV file submitted with the code repository. A sample of the CSV format is shown below:

```
Image,Generator Output,Verifier Output,Similarity,Is Fault
img_1.jpg,two cats sleeping on a couch,two cats sleeping on a couch with a remote,0.917,False
img_2.jpg,a kitchen with a table and chairs in it,the kitchen has all white appliances and a t
...
```